



ТОМСКИЙ
ПОЛИТЕХНИЧЕСКИЙ
УНИВЕРСИТЕТ



ИНЖЕНЕРНАЯ ШКОЛА
ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ
И РОБОТОТЕХНИКИ

ШИФРЫ ЗАМЕНЫ

Курс «Безопасность
информационных систем», ЛР 1

Фаерман Владимир Андреевич,
ИСИБ ТУСУР

2025/2026

СОДЕРЖАНИЕ РАБОТЫ



ИНЖЕНЕРНАЯ ШКОЛА
ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ
И РОБОТОТЕХНИКИ

«В мире криптографии существуют два типа шифров. Первый защитит вашу переписку от младшей сестрёнки. Второй защитит вашу переписку от правительственных спецслужб»

Брюс Шнайер,
ведущий криптолог в IBM,
профессор
Университет Гарварда



- Виды шифров, классификация
- Русский телеграфный алфавит
- Шифр Цезаря, шифр Тритимиуса
- Полиалфавитные шифры замены (шифр Виненера, усиленный шифр Тритимуса)
- Задание на лабораторную работу
- Требования к отчёту
- Требования к защите

ВИДЫ ШИФРОВ



- **Шифр** – система *обратимых* преобразований, зависящая от некоторого *секретного параметра* (ключа) и предназначенная для обеспечения секретности передаваемой информации.
- **Ключ** – секретная информация, которая используется криптографическим алгоритмом при зашифровании или расшифровании сообщений. Если один и тот же ключ используется как для шифрования, так и расшифрования, то такую криптосистему называют *симметричной*. Если используются разные ключи, то *асимметричной*.
- **Шифры замены** – это метод шифрования, который производит *замену* каждой буквы открытого текста на какой-то символ шифрованного текста. Получатель сообщения расшифровывает его путём обратной замены.
- **Шифры перестановки** – это метод шифрования, в котором последовательность элементов исходного открытого текста изменяется, при переносе в шифрованный текст, в соответствии с некоторым правилом перестановки. Получатель сообщения при расшифровании совершает обратную перестановку элементов сообщения.
- **Комбинированные шифры** – это метод шифрования, который сочетает в себе замены и перестановки.

ВИДЫ ШИФРОВ



- **Шифры простой замены** – разновидность шифров замены, в которых *каждый* уникальный *СИМВОЛ* открытого текста заменяется на *один и тот же* уникальный *СИМВОЛ* шифротекста.
- **Шифры омофонной замены** – разновидность шифров замены, в которых *каждый* уникальный *СИМВОЛ* открытого текста заменяется на *один* или *несколько наборов* символов шифротекста (например, «о» может заменяться, например, на «еоо», «щуг» и другие варианты). Число возможных альтернативных вариантов замены пропорционально частоте заменяемого символа («о» - самая частотная буква кириллицы и поэтому ей соответствует наибольшее число вариантов замены; буква «щ» - наименее частотная, поэтому ей соответствует один набор символов, например, «бгы»).
- **Полиалфавитные шифры замены** – разновидность шифров замены, в которых *каждый* уникальный *СИМВОЛ* открытого текста может (потенциально) заменяться *более чем на один* символ шифротекста. Полиалфавитное шифрование представляет собой *несколько шифров простой замены*, которые применяются к символам текста поочередно.

ВИДЫ ШИФРОВ



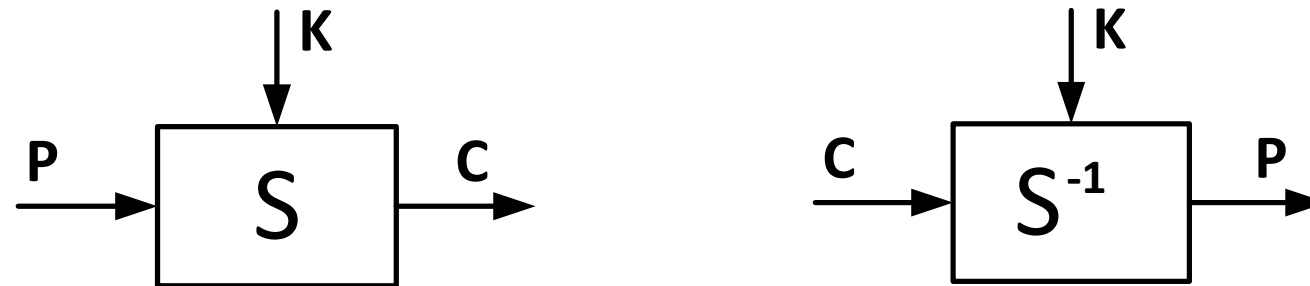
- **Потоковые шифры** – форма применения методов шифрования, при которой открытый текст посимвольно преобразуется в шифротекст.
- **Блочный шифры** – форма применения методов шифрования, при которой открытый текст преобразуется в шифротекст поблочно. Блок – элемент текста, который состоит из фиксированного числа символов.
- **Блочный полиалфавитный шифр замены** – полиалфавитный шифр замены, который применяется для поблочного шифрования текста. Лабораторная работа направлена на программную реализацию такого шифра.
- **S-блок** – от англ. *substitution block*, алгоритмический примитив, который преобразует блок открытого текста фиксированной длины в блок шифротекста той же длины.

РУССКИЙ ТЕЛЕГРАФНЫЙ АЛФАВИТ



ИНЖЕНЕРНАЯ ШКОЛА
ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ
И РОБОТОТЕХНИКИ

Мы будем рассматривать **зашифрование** и **расшифрование** следующим образом:



- **P** – сообщение в открытом тексте (от *plaintext*);
- **C** – сообщение в шифротексте (от *ciphertext*);
- **S(I, K)** – операция зашифрования текста на входе (I) с применением ключа K с использованием шифра замены (от *substitutional cipher*);
- **S⁻¹(I, K)** – операция расшифрования текста на входе (I) с применением ключа (K) с использованием шифра замены.

Мы будем считать, что и открытый текст (P), и шифротекст (C), и ключ (K) используют **одинаковый алфавит** (это тривиальное допущение, поскольку произвольную битовую строку при желании можно представить в виде массива символов из произвольного алфавита, при условии что ему соответствует равномерный безызбыточный код). Для определённости и наглядности мы будем использовать **русский телеграфный алфавит**.

РУССКИЙ ТЕЛЕГРАФНЫЙ АЛФАВИТ



ИНЖЕНЕРНАЯ ШКОЛА
ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ
И РОБОТОТЕХНИКИ

Русский телеграфный алфавит – представляет собой основные буквы русского языка (31 буква, без «ё» и «ъ») и пробел, то есть всего 32 символа. Знаки препинания кВ первой половине 20-ого века пришёл на смену алфавиту Морзе для автоматических телеграфов (телетайп). Как это работало:

- вместо «ё» использовался «е», вместо «ъ» использовался «ь»;
- знаки препинания описывались специальными словами («тчк», «зпт» и др.).

№	Символ	Код
1	А	00001
2	Б	00010
3	В	00011
4	Г	00100
5	Д	00101
6	Е	00110
7	Ж	00111
8	З	01000

№	Символ	Код
9	И	01001
10	Й	01010
11	К	01011
12	Л	01100
13	М	01101
14	Н	01110
15	О	01111
16	П	10000

№	Символ	Код
17	Р	10001
18	С	10010
19	Т	10011
20	У	10100
21	Ф	10101
22	Х	10110
23	Ц	10111
24	Ч	11000

№	Символ	Код
25	Ш	11001
26	Щ	11010
27	Ы	11011
28	Ь	11100
29	Э	11101
30	Ю	11110
31	Я	11111
32	—	00000

РУССКИЙ ТЕЛЕГРАФНЫЙ АЛФАВИТ



ИНЖЕНЕРНАЯ ШКОЛА
ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ
И РОБОТОТЕХНИКИ

Мы будем считать, что над символами алфавита определены простые *арифметические операции*. Строго говоря, они подчиняется законам *арифметики над конечными полями*. Просто говоря, они подчиняются следующим правилам.

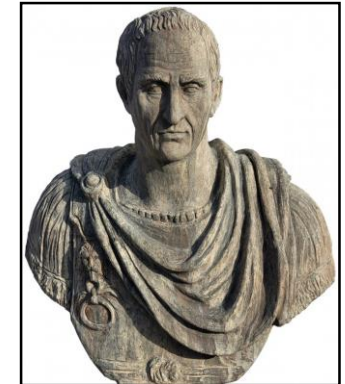
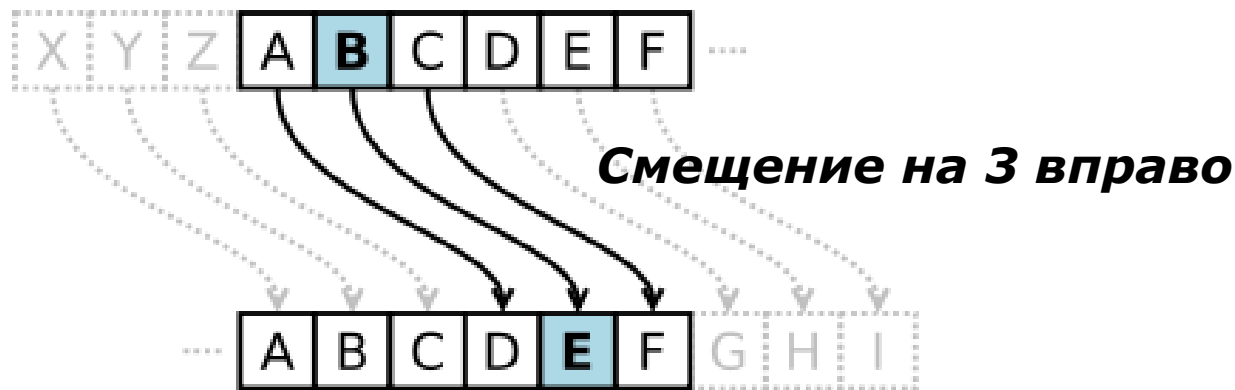
- Пусть X, Y, Z это символы алфавита, а $n(X), n(Y), n(Z)$ это десятичные порядковые номера этих символов. Операцию обратную $x=n(X)$ обозначим $X=d(x)$, где x – число от 0 до 31.
- Операция $Z=X+Y$ означает, что необходимо найти такой символ Z , который в таблице находится на $n(Y)$ позиций после X . Если при этом мы выходим за пределы таблицы с конца, то мы продолжаем поиск с её начала.
Например, «А»+«З»=«И»; «Щ»+«З»=«Б». Формально, $n(Z)=\text{mod}[n(X)+n(Y), 32]$.
- Операция $Z=X-Y$ означает, что необходимо найти такой символ Z , который в таблице находится на $n(Y)$ позиций до X . Если при этом мы выходим за пределы таблицы с начала, то мы продолжаем поиск с её конца.
Например, «Щ»-«З»=«С»; «А»-«З»=«Ш». Формально, $n(Z)=\text{mod}[32+n(X)-n(Y), 32]$.
- Легко доказать, что если $Z=X+Y-Y$, то $Z=X$. То есть операции сложения и вычитания являются *взаимно обратными*.

КЛАССИЧЕСКИЙ ШИФР ЦЕЗАРЯ

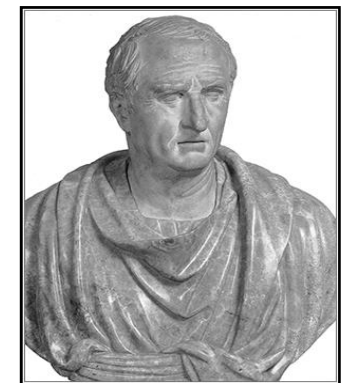


ИНЖЕНЕРНАЯ ШКОЛА
ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ
И РОБОТОТЕХНИКИ

- **Шифр Цезаря** – исторический шифр простой замены, следы применения которого сохранились до наших дней в переписке проконсула (соправитель, наместник в Галлии и Италии) Цезаря и Цицероном легатом (чиновник, уполномоченный сенатом на переговоры) консула Рима в период от 60-ых до 50-ых годов до н.э.
- **Шифр Цезаря** – шифр, при использовании которого каждая буква из открытого текста заменяется на такую букву, которая в алфавите находится на некотором *постоянном числе позиций правее* от рассматриваемой буквы. Число сдвигаемых позиций считается ключом шифра.



Гай Юлий Цезарь,
диктатор республики



Марк Туллий Цицерон,
сенатор и легат, философ

КЛАССИЧЕСКИЙ ШИФР ЦЕЗАРЯ



ИНЖЕНЕРНАЯ ШКОЛА
ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ
И РОБОТОТЕХНИКИ

Формально, *шифр Цезаря* исключительно прост алгоритмически.

- Пусть $S(P, K)$ – зашифрование с использованием шифра Цезаря. Считаем, для удобства дальнейших рассуждений, что K – одиночный символ.
- Считаем, что $p \in P$, $c \in C$ (p и c одиночные символы открытого текста и шифротекста соответственно).
- Тогда $C = S(P, K)$, следовательно $c = S(p, K) \forall p$. Отсюда $c = d(\text{mod}[n(p) + n(K), 32]) \forall p$. То есть мы повторяем одну и ту же операцию для каждого символа в исходном тексте.
- Аналогично для каждого символа шифротекста. Имеем $P = S^{-1}(C, K)$, следовательно $p = S^{-1}(c, K) \forall p$. Отсюда $p = d(\text{mod}[32 + n(p) - n(K), 32]) \forall p$.

Рассмотрим примеры.

- $P = \text{«СЫЗРАНЬ»}$, $K = \text{«А»} \Rightarrow C = \text{«ТЫСБОЭ»}$; $P = \text{«СЫЗРАНЬ»}$, $K = \text{«Ф»} \Rightarrow C = \text{«ЖПЭХГР»}$
- $P = \text{«ГЛАДИУС»}$, $K = \text{«Е»} \Rightarrow C = \text{«ЙСЖКОЩЧ»}$;
 $P = \text{«ГЛАДИОЛУС»}$, $K = \text{«Е»} \Rightarrow C = \text{«ЙСЖКОФСЩЧ»}$.

КЛАССИЧЕСКИЙ ШИФР ТРИТИМУСА



ИНЖЕНЕРНАЯ ШКОЛА
ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ
И РОБОТОТЕХНИКИ

- **Шифр Тритимуса** – шифр простой замены, который предложил германский католический мыслитель Иохан из Триттенхайма (ныне земля Саар), более известный как аббат Тритимус. Шифр был разработан и математически исследован в трудах «Стеганография» (1500) и «Полиграфия» (1508).
- **Шифр Тритимуса** – использует тот же алгоритмический принцип, что и шифр Цезаря. Однако секретом является *не число позиций в сдвиге, а последовательность символов* в используемом алфавите. Сдвиг осуществляется на 8 символов, что связано с практикой применения шифровальных таблиц.
- **Ключ** – вместо обычного алфавита, алгоритмом используется *модифицированный алфавит*, который является секретом. Рабочий алфавит строится непосредственно *перед зашифрованием* (расшифрованием) с помощью ключа (ключевой фразы).



Иоган Тритимус,
аббат (настоятель) монастыря
в Вюрцбурге (Шотландия)

КЛАССИЧЕСКИЙ ШИФР ТРИТИМУСА



ИНЖЕНЕРНАЯ ШКОЛА
ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ
И РОБОТОТЕХНИКИ

Рассмотрим принцип построения рабочего алфавита для *шифра Тритимуса*

- Пусть у нас есть ключевая фраза (или слово) *К*. Исторически выбирались длинные слова, которые легко запомнить.
- В *рабочий алфавит*, в первую очередь, включаются те символы, которые присутствуют в ключевой фразе. Причём в порядке их упоминания.
- Все символы, которые не были представлены в ключевой фразе, вносятся в *рабочий алфавит* в обычном (алфавитном) порядке.
- Для удобства ручного применения, *рабочий алфавит* вносится в таблицу. Для нас подходит 4x8 (32 ячейки, по числу символов).

КЛАССИЧЕСКИЙ ШИФР ТРИТИМУСА



Пусть нам дано, например, ключевое слово «ВЫСОКОПРЕВОСХОДИТЕЛЬСТВО». Сформируем рабочий алфавит, используя этот ключ.

«Голова» рабочего алфавита (символы взяты из ключа)

В	Ы	С	О	К	П	Р	Е
Х	Д	И	Т	Л	Ь	А	Б
Г	Ж	З	Й	М	Н	У	Ф
Ц	Ч	Ш	Щ	Э	Ю	Я	—

«Хвост» рабочего алфавита (символы взяты из обычного алфавита)

Таблица заполняется обычным образом (сверху-вниз и слева-направо). Порядковый номер символа в *рабочем алфавите* тривиально определяется и однозначно следует из его положения в таблице.

КЛАССИЧЕСКИЙ ШИФР ТРИТИМУСА



Для *зашифрования* используется сдвиг *вперёд* на 8 позиций. То есть берётся символ, находящийся непосредственно заменяемым (или в первой строке, если заменяемый находится в последней). При *расшифровании* берётся символ находящийся над исходным (или в последней строке, если текущий символ шифротекста находится в первой строке).

Символы открытого текста

Шифрование ↓	В	Ы	С	О	К	П	Р	Е	↑ Расшифрование
	Х	Д	И	Т	Л	Ь	А	Б	
	Г	Ж	З	Й	М	Н	У	Ф	
	Ц	Ч	Ш	Щ	Э	Ю	Я	—	

Символы шифротекста

Рассмотрим примеры.

- Р=«СЫЗРАНЬ», К=«ВЫСОКОПРЕВОСХОДИТЕЛЬСТВО» => С=«ИДШАУЮН».
- Р=«ГЛАДИУС», К=«ВЫСОКОПРЕВОСХОДИТЕЛЬСТВО» => С=«ЦМУЖЗЯИ»;
Р=«ГЛАДИОЛУС», К=«ВЫСОКОПРЕВОСХОДИТЕЛЬСТВО» => С=«ЦМУЖЗТМЯИ».

ПОЛИАЛФАВИТНЫЕ ШИФРЫ



- **Полиалфавитные шифры замены** – идея состоит в том, что при применении правила замены, используется *не единственный* секрет для всех символов, а набор таких секретов. При этом, секреты из набора (*элементарные ключи*) применяются *поочерёдно* для шифрования символов открытого текста.
- **Элементарный ключ** – секрет, который применяется для шифрования отдельного символа. *Элементарные ключи* формируются из *основного ключа*, который обычно представляет собой ключевую фразу (или слово). Предполагается, что из *основного ключа* можно получить сколь угодно много *элементарных ключей*, однако, не все из них будут уникальны (начнут повторяться). Операцию получения следующего *элементарного ключа* из *основного ключа* будем называть **сдвигом**.

Введём следующие обозначения

- K – основной ключ; $k(i)$ – элементарный ключ; i – число сдвигов, путём которых элементарный ключи $k(i)$ получен из основного ключа K .

ПОЛИАЛФАВИТНЫЙ ШИФР ЦЕЗАРЯ (ШИФР ВИЖЕНЕРА)



ИНЖЕНЕРНАЯ ШКОЛА
ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ
И РОБОТОТЕХНИКИ

- К каждому символу в отдельности (для зашифрования и расшифрования) применяется ранее описанный алгоритм *простого шифра Цезаря*.
- В качестве *основного ключа* используется ключевая фраза (или слово).
- *Элементарный ключ* получается с помощью *основного ключа* путём простых манипуляций. Например, следующим образом: $k(i) = K(q)$, где $K(q)$ – q -ый символ основного ключа, где $q = \text{mod}[i, L(K)]$, $L(K)$ – длина основного ключа. *Начальное значение* элементарного ключа $k(0) = \text{«}_\text{»}$.
- Перед шифрованием очередного символа с помощью *элементарного ключа* производится сдвиг, по алгоритму описанному выше. До начала работы с текстом, могут производиться «холостые» сдвиги.
- Версия сдвига с более сложным подходом к формированию элементарного ключа, например где $k(i) = K(x)$ при $x = \text{mod} [(K(q) \times c + i), 32]$ называется *аффинным сдвигом*, где c – заданный коэффициент.



Блез де Виженер,
придворный учёный,
посол Франции в Италии
(в 1560-ые гг.)

СДВИГ КЛЮЧА В ШИФРЕ ТРИТИМУСА



ИНЖЕНЕРНАЯ ШКОЛА
ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ
И РОБОТОТЕХНИКИ

- К каждому символу в отдельности (для зашифрования и расшифрования) применяется ранее описанный алгоритм *простого шифра Тритимуса*.
- В качестве *основного ключа* используется ключевая фраза (или слово). По ней строится таблица подстановки.
- При сдвиге строится таблица для строки, которая является периодическим продолжением *основного ключа*, то есть его конкатенации с самим собой.
- Для того, чтобы при сдвиге перестраивать таблицу «с нуля», проще передавать текущую версию таблицы и построить алгоритм её модификации.
- Описанный способ получения *элементарных ключей* используется нами для примера (мне неизвестно, чтобы кто-то предлагал его в таком виде). Это сегодня связано с тем, что шифр Тритимуса представляет скорее историческую ценность, чем практическую.

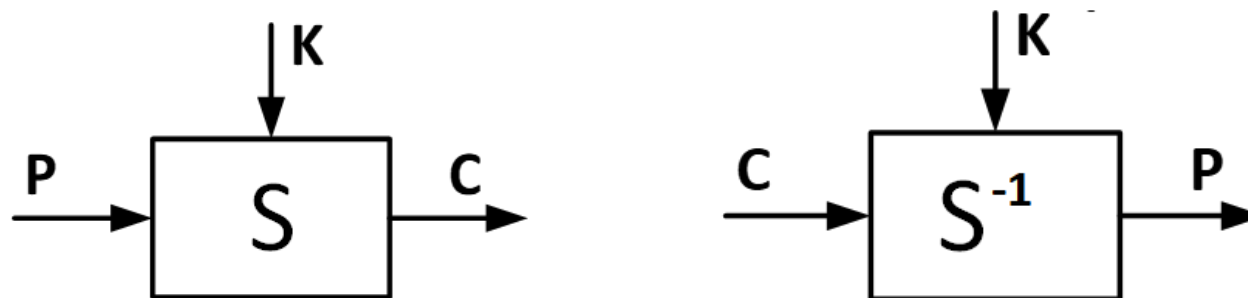


Николло Макиавели,
госсекретарь Флоренции,
философ-реалист
(в 1500-ые гг.)

ШИФРОВАНИЕ ПО БЛОКАМ



Для удобства дальнейшей работы (в рамках последующих лабораторных) нам потребуется реализовать *унифицированный интерфейс* к шифру замены. Для этого опишем требования ко входам и выходам S-блоков.



Код элемента	Пояснение	Тип данных	Размер массива
P	Открытый текст	Массив символов [целых чисел от 0 до 31]	4 символа [точно]
C	Шифротекст		4 символа [точно]
K	Основной ключ		16 символов [точно]

ПРИЁМЫ ПО УСИЛЕНИЮ



- Для повышения качества S-блока введём в него операцию, осуществляющую неоднозначное (зависимое от ключа) перемешивание символов
- Введём следующую заплатку, которая будет применяться к блоку после блочного шифрования:
 - Пусть E_1, E_2, E_3, E_4 – символы зашифрованного сообщения до модификации, а $K_1, \dots, K_{16}$ символы ключа. Введём $q = (n(K_1) - n(K_2) + \dots - n(K_{16}))$ и определим $q1 = \text{mod}(q, 4)$, $q = \text{div}(q, 4)$; $q2 = \text{mod}(q, 3)$, $q = \text{div}(q, 3)$; $q3 = \text{mod}(q, 2)$.
 - В зависимости от значений $q1, q2, q3$ мы определим порядок суммирования символов $E = \{E_1, E_2, E_3, E_4\}$. Для этого мы выполняем $E[q2] = E[q2] + E[q1]$, $E[q3] = E[q3] + E[q2]$, $E[q4] = E[q4] + E[q3]$, $E[q1] = E[q1] + E[q4]$.
 - После такой манипуляции, изменение одного символа открытого текста приведёт к изменению хотя бы двух символов шифротекста. Для того, чтобы изменения гарантировано присутствовали во всех символах, можно процедуру повторить дважды, причём лучше раз до основной функции и один раз после.
 - При расшифровании действия производятся в обратном порядке.



ТЕСТЫ НА НАДЕЖНОСТЬ

- Чтобы S-блок мог использоваться необходимо, чтобы он был чувствительным к изменениям во входном блоке и в ключе:
 1. **Тест на малое изменение входа.** Положим, что P и P' близкие входы (один из символов изменён на соседний), K – ключ, тогда $C1=S(P,K)$, $C2=S(P',K)$. Разница между $C1$ и $C2$ должна быть значительной (не сводится к изменению одного символа).
 2. **Тест на ротацию входа.** Положим, что P и P' ротированные входы (состоят из одних и тех же символов, их позиции изменены), K – ключ, тогда $C1=S(P,K)$, $C2=S(P',K)$. Выходы $C1$ и $C2$ не должны состоять из одних и тех же символов.
 3. **Тест на линейность преобразования по входу.** Положим, что $P1$, $P2$ и $P3$ разные входы, а K – ключ. Причём $P3 = P1 + P2$. Положим что $C1=S(P1,K)$, $C2=S(P2,K)$, $C3=S(P3,K)$. Между $(C1 + C2)$ и $C3$ должна быть существенная разница.
 4. Аналогичные тесты (3 штуки) можно провести применительно к ключу.
 5. Обычно делается случайная выборка входов (для нас, хотя бы 20-30), ключей (для нас, хотя бы 10-20) и они тестируются во всех комбинациях. Оценивается доля полностью пройденных тестов, частично пройденных тестов, заваленных тестов. Анализируя статистику делают вывод о наличии слабых ключей, неудобных входов (для которых доля проваленных тестов сильно хуже ожидаемой).

Хороший S-блок должен проходить все тесты, быть вычислительно простым. Наши не такие.

ЗАДАНИЕ



- 1. Провести подготовительную работу.** Реализовать структуру данных, для хранения символов русского телеграфного алфавита (5 битовые числа). Описать функции получения символа по значению (от 0 до 31), значения по символу. Описать функции суммирования и вычитания символов.
- 2. Реализовать преобразование одиночного символа.** В зависимости от выбранного варианта, реализовать шифр Цезаря или шифр Тритимуса. Проверить работоспособность функции зашифрования символа, расшифрования символа (с использованием различных ключей). Ключом считать слово (для шифра Цезаря использовать первый символ).
- 3. Реализовать шифрование текста.** Реализовать программную обёртку на функции, работающие с одиночными символами. На вход поступает блок текста, преобразование применяется последовательно к каждому из них. На выходе – блок из того же числа символов. Проверить работоспособность, применительно к зашифрованию и расшифрованию блоков (использовать различные ключи).
- 4. Реализовать полиалфавитные шифры.** Доработать функции шифрования и расшифрования, таким образом, чтобы применялись полиалфавитные варианты шифров.
- 5. Подготовить обёртку для шифрования блоков по 4 символа.** Проверить работоспособность S-блоков (прямого и обратного).
- 6. Применить приёмы для усиления S-блоков.** Реализовать операцию смешивания символов в блоке зависимую от ключа.
- 7. Провести исследования S-блоков.** Оценить надёжность S-функции до модификации и после неё. Сделать выводы и обосновать их.
- 8. Подготовить отчёт по лабораторной работе.** Требования представлены далее.

ТРЕБОВАНИЯ К ОТЧЁТУ



- 1. Введение.** Цели лабораторной работы. Задачи лабораторной работы. Индивидуальные цели и задачи (которые группа ставит перед собой).
- 2. Теоретическая часть.** Любые теоретические сведения в достаточном объёме (их можно будет использовать на защите). Требования по оригинальности этого раздела нет.
- 3. Практическая часть.** Последовательно описать решение каждого из этапов создания блочного шифра замены. Необходимо приводить диаграммы, иллюстрирующие особенности реализации программного решения (диаграммы классов (UML), диаграммы последовательности вызовов (Sequence diagram) для S-блока, блок-схемы).
- 4. Аналитическая часть.** Описать верификацию (проверку работоспособности) созданной программы. Обосновать все принятые командой решения. При наличии тестировщика, набор дополнительных тестов разработать самостоятельно. Убедиться, что шифрование чувствительно ключу. Исследовать влияние минимальных отличий в ключе на результат расшифрования. Привести соображения относительно надёжности созданной криптографической системы (с операцией усиления и без неё).
- 5. Описание вклада участников группы.** Описать роли и функции всех участников в группе. Составить матрицу ответственности RACI. Задачи не должны дублироваться (например, если программистов было двое, то детализировать вклад каждого из них в код). Типичные роли – программист, лидер команды, тестировщик, системный аналитик.
- 6. Заключение.** Привести выводы по проделанной работе (включая результаты анализа). Описать индивидуальные итоги работы (что для себя извлекли, понравилось или нет).

ТРЕБОВАНИЯ К ЗАЩИТЕ



ИНЖЕНЕРНАЯ ШКОЛА
ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ
И РОБОТОТЕХНИКИ

- **Лабораторные работы защищаются в индивидуальном порядке.** Каждый участник группы защищает лабораторную индивидуально и оценивается независимо. На защиту представляется групповой отчёт (копия «в бумаге» может использоваться всеми при подготовке ответов на вопросы).
- **Предмет защиты.** На защите обсуждаются следующие вопросы: общие теоретические сведения, все результаты групповой работы (код, анализ, отчёт), индивидуальный вклад в групповую работу.
- **Защиты не обязательны.** В ряде случаев, работа может быть зачтена по факту представления отчёта (всей группе или её отдельных членов, вклад которых в работу не вызывает сомнений). Во внимание принимаются следующие факторы: качество проделанной работы (программы, анализа), качество подготовки отчёта, посещаемость и активность участников группы на аудиторных занятиях.

- **Созданные S-блоки - игрушечные.** Несмотря на то, что мы приняли какие-то рациональные меры по их усилению, наши S-блоки всё равно далеки от того, что ожидается от современных «боевых» решений. Сам подход, который мы использовали при их создании – наивный и отличается от правильного.
- **Маленький качественный S-блок лучше.** В современных решениях обычно используются 4 (редко), 8 (почти всегда) или 12 (редко) битовые S-блоки. Это связано с тем, что маленькие блоки проще детально исследовать на предмет надёжности, проще без сложных вычислений добиться качественного прохождения тестов.
- **Статичный S-блок проще чем динамический.** В нашем случае блоки динамические, то есть ключ определяет карту преобразований, которые производит блок. Создание и хранение такой карты, даже для небольших блоков, затратно по вычислениям и памяти. Рациональнее хранить статичную (не зависящую от ключа) карту преобразований $S(x)$, а ключ применять отдельно, например $E(x, K) = S(x+K)+K$. Такой подход сочетает вычислительную простоту, доказанную надёжность и функциональную предсказуемость.



ТОМСКИЙ
ПОЛИТЕХНИЧЕСКИЙ
УНИВЕРСИТЕТ



ИНЖЕНЕРНАЯ ШКОЛА
ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ
И РОБОТОТЕХНИКИ

СПАСИБО ЗА ВНИМАНИЕ!

Курс «Безопасность
информационных систем», ЛР 1

Фаерман Владимир Андреевич,
ИСИБ ТУСУР (fva@fb.tusur.ru)

2025/2026

ПРИЛОЖЕНИЕ - ОБЩЕЕ



Символ в число

```
sym2num(sym_in) := || tmp ← str2vec(sym_in)
|| if tmp_0 ≠ 95
|| || out ← tmp_0 - 1039 - if tmp_0 > 1066
|| || || 1
|| || || else
|| || || 0
|| || else
|| || 0
```

$\text{sym2num}(\text{"O"}) = 15$ $\text{sym2num}(\text{"Ж"}) = 7$

Текст в массив чисел

```
text2array(TXT_IN) := || for i ∈ 0, 1..strlen(TXT_IN)-1
|| || tmp ← substr(TXT_IN, i, 1)
|| || out_i ← sym2num(tmp)
|| out
```

$\text{IN_TEXT} := \text{"АБВГДЕЖЗИЙКЛМНОПРСТУФХЦЧШЩЬЬЪЮЯ_"}$

$\text{OUT_ARR} := \text{text2array}(\text{IN_TEXT})^T = [1 \ 2 \ 3 \ 4 \ 5 \ 6 \ 7 \ 8 \ 9 \ 10 \ 11 \ 12 \ 13 \ 14 \ 15 \ 16 \ 17 \ 18 \ 19 \ 20 \ 21 \ 22 \ 23 \ 24 \ 25 \ 26 \ 27 \ 28 \ 29 \ 30 \ 31 \ 0]$

$\text{OUT_TEXT} := \text{array2text}(\text{OUT_ARR}^T) = \text{"АБВГДЕЖЗИЙКЛМНОПРСТУФХЦЧШЩЬЬЪЮЯ_"}$

Число в символ

```
num2sym(num_in) := || if num_in ≠ 0
|| || tmp_0 ← num_in + 1039 + if num_in > 26
|| || || 1
|| || || else
|| || || 0
|| || out ← vec2str(tmp)
|| else
|| || " -"
```

$\text{num2sym}(7) = \text{"Ж"}$ $\text{num2sym}(14) = \text{"H"}$

Массив чисел в текст

```
array2text(ARR_IN) := || out ← ""
|| for i ∈ 0, 1..length(ARR_IN)-1
|| || out ← concat(out, num2sym(ARR_IN_i))
|| out
```

ПРИЛОЖЕНИЕ - ОБЩЕЕ



Сложение символов

$$\text{add_s}(s1_in, s2_in) := \begin{cases} tmp \leftarrow \text{sym2num}(s1_in) + \text{sym2num}(s2_in) \\ out \leftarrow \text{num2sym}(\text{mod}(tmp, 32)) \end{cases}$$

$$\text{add_s}(\text{“Я”}, \text{“Ж”}) = \text{“Е”} \quad \text{sub_s}(\text{“Е”}, \text{“Ж”}) = \text{“Я”}$$

$$\text{add_txt}(T1_IN, T2_IN) := \begin{cases} out \leftarrow \text{“”} \\ m \leftarrow \min(\text{strlen}(T1_IN), \text{strlen}(T2_IN)) \\ \text{if } \text{strlen}(T1_IN) > \text{strlen}(T2_IN) \\ \quad T_IN \leftarrow T1_IN \\ \text{else} \\ \quad T_IN \leftarrow T2_IN \\ M \leftarrow \text{strlen}(T_IN) \\ \text{for } i \in 0, 1..m-1 \\ \quad t1 \leftarrow \text{substr}(T1_IN, i, 1) \\ \quad t2 \leftarrow \text{substr}(T2_IN, i, 1) \\ \quad out \leftarrow \text{concat}(out, \text{add_s}(t1, t2)) \\ \text{if } M > m \\ \quad \text{for } i \in m, m+1..M-1 \\ \quad \quad t \leftarrow \text{substr}(T_IN, i, 1) \\ \quad \quad out \leftarrow \text{concat}(out, t) \\ out \end{cases}$$

$$TT1 := \text{“ЕЖИК”} \quad TT2 := \text{“В_ТУМАНЕ”}$$

$$TT5 := \text{add_txt}(TT1, TT2) = \text{“ИЖЬЯМАНЕ”}$$

$$\text{add_txt}(TT6, TT4) = \text{“БАРОН”}$$

Вычитание символов

$$\text{sub_s}(s1_in, s2_in) := \begin{cases} tmp \leftarrow \text{sym2num}(s1_in) - \text{sym2num}(s2_in) + 32 \\ out \leftarrow \text{num2sym}(\text{mod}(tmp, 32)) \end{cases}$$

$$\text{sub_txt}(T1_IN, T2_IN) := \begin{cases} out \leftarrow \text{“”} \\ m \leftarrow \min(\text{strlen}(T1_IN), \text{strlen}(T2_IN)) \\ \text{if } \text{strlen}(T1_IN) > \text{strlen}(T2_IN) \\ \quad T_IN \leftarrow T1_IN \\ \quad flag \leftarrow 0 \\ \text{else} \\ \quad T_IN \leftarrow T2_IN \\ \quad flag \leftarrow 1 \\ M \leftarrow \text{strlen}(T_IN) \\ \text{for } i \in 0, 1..m-1 \\ \quad t1 \leftarrow \text{substr}(T1_IN, i, 1) \\ \quad t2 \leftarrow \text{substr}(T2_IN, i, 1) \\ \quad out \leftarrow \text{concat}(out, \text{sub_s}(t1, t2)) \\ \text{if } M > m \\ \quad \text{for } i \in m, m+1..M-1 \\ \quad \quad t \leftarrow \text{substr}(T_IN, i, 1) \\ \quad \quad \text{if } flag = 1 \\ \quad \quad \quad out \leftarrow \text{concat}(out, \text{sub_s}(\text{“_”}, t)) \\ \quad \quad \text{else} \\ \quad \quad \quad out \leftarrow \text{concat}(out, \text{sub_s}(t, \text{“_”})) \\ out \end{cases}$$

$$TT3 := \text{“БАРОН”} \quad TT4 := \text{“ВАРАН”}$$

$$TT6 := \text{sub_txt}(TT3, TT4) = \text{“Я_Н_”}$$

$$\text{sub_txt}(TT5, TT2) = \text{“ЕЖИК_”}$$

$$\text{sub_txt}(TT5, TT1) = \text{“В_ТУМАНЕ”}$$

ПРИЛОЖЕНИЕ - ЦЕЗАРЬ



```
frw_Cesar(TXT_IN, KEY_IN) := || out ← ""  
                             || key ← substr(KEY_IN, 0, 1)  
                             || for i ∈ 0, 1..strlen(TXT_IN) - 1  
                             ||   || tmp ← substr(TXT_IN, i, 1)  
                             ||   || out ← concat(out, add_s(tmp, key))  
                             || out
```

IN := "ОЛОЛО_КРИНЖ" K1 := "_" K2 := "X"

OUT1 := frw_Cesar(IN, K1) = "ОЛОЛО_КРИНЖ"
OUT2 := frw_Cesar(IN, K2) = "ДБДБДХАЖЯГЭ"

```
inv_Cesar(TXT_IN, KEY_IN) := || out ← ""  
                             || key ← substr(KEY_IN, 0, 1)  
                             || for i ∈ 0, 1..strlen(TXT_IN) - 1  
                             ||   || tmp ← substr(TXT_IN, i, 1)  
                             ||   || out ← concat(out, sub_s(tmp, key))  
                             || out
```

IN1 := inv_Cesar(OUT1, K1) = "ОЛОЛО_КРИНЖ"
IN2 := inv_Cesar(OUT2, K2) = "ОЛОЛО_КРИНЖ"

Полиалфавитный шифр Цезаря (шифр Виженера)

```
frw_poly_Cesar(TXT_IN, KEY_IN) := || out ← ""  
                                  || t_k ← "_"  
                                  || K ← strlen(KEY_IN)  
                                  || for i ∈ 0, 1..strlen(TXT_IN) - 1  
                                  ||   || t_i ← substr(TXT_IN, i, 1)  
                                  ||   || q ← mod(i, K)  
                                  ||   || t_k ← add_s(t_k, substr(KEY_IN, q, 1))  
                                  ||   || out ← concat(out, add_s(t_i, t_k))  
                                  || out
```

IN := "ОЛОЛО_КРИНЖ" K1 := "X" K2 := "ПАНТЕОН"

OUT1 := frw_poly_Cesar(IN, K1) = "ДЧРГЭГДАОЙШ"
OUT2 := frw_poly_Cesar(IN, K2) = "ЯЭНЮЖЖ_ХОБН"

```
inv_poly_Cesar(TXT_IN, KEY_IN) := || out ← ""  
                                  || t_k ← "_"  
                                  || K ← strlen(KEY_IN)  
                                  || for i ∈ 0, 1..strlen(TXT_IN) - 1  
                                  ||   || t_i ← substr(TXT_IN, i, 1)  
                                  ||   || q ← mod(i, K)  
                                  ||   || t_k ← add_s(t_k, substr(KEY_IN, q, 1))  
                                  ||   || out ← concat(out, sub_s(t_i, t_k))  
                                  || out
```

IN1 := inv_poly_Cesar(OUT1, K1) = "ОЛОЛО_КРИНЖ"
IN2 := inv_poly_Cesar(OUT2, K2) = "ОЛОЛО_КРИНЖ"

ПРИЛОЖЕНИЕ - ЦЕЗАРЬ



ИНЖЕНЕРНАЯ ШКОЛА
ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ
И РОБОТОТЕХНИКИ

$reverse_str("ЗОЛОТО") = "ОТОЛОЗ"$ $reverse_str("ОТОЛОЗ") = "ЗОЛОТО"$

S-блоки для шифра Цезаря

```

frw_S_Caesar(BLOCK_IN, KEY_IN) :=
    out ← "input_error"
    if (strlen(BLOCK_IN) = 4) ∧ (strlen(KEY_IN) = 16)
        KEY_TMP ← "____"
        C ← [1 -1 1 2 -2 1 1 3 -1 2]T
        out ← BLOCK_IN
        KEY_EXT ← concat(KEY_IN, KEY_IN)
        for i ∈ 0, 1..7
            S_TMP ← substr(KEY_EXT, i·2, 4)
            B_TMP ← text2array(S_TMP)
            for k ∈ 0, 1..3
                x ← mod(2·i + k, 10)
                A_TMPk ← mod(64 + k + Cx · B_TMPk, 32)
            KEY_TMP ← add_txt(KEY_TMP, array2text(A_TMP))
        out ← frw_poly_Caesar(out, KEY_TMP)
    out
    
```

```

inv_S_Caesar(BLOCK_IN, KEY_IN) :=
    out ← "input_error"
    if (strlen(BLOCK_IN) = 4) ∧ (strlen(KEY_IN) = 16)
        KEY_TMP ← "____"
        C ← [1 -1 1 2 -2 1 1 3 -1 2]T
        out ← BLOCK_IN
        KEY_EXT ← concat(KEY_IN, KEY_IN)
        for i ∈ 0, 1..7
            S_TMP ← substr(KEY_EXT, i·2, 4)
            B_TMP ← text2array(S_TMP)
            for k ∈ 0, 1..3
                x ← mod(2·i + k, 10)
                A_TMPk ← mod(64 + k + Cx · B_TMPk, 32)
            KEY_TMP ← add_txt(KEY_TMP, array2text(A_TMP))
        out ← inv_poly_Caesar(out, KEY_TMP)
    out
    
```

K1 := "ХОРОШО_БЫТЬ_ВАМИ" K2 := "ЧЕРНОВОЙ_АХИЛЛЕС"

IN1 := "БЛОК" OUT11 := frw_S_Caesar(IN1, K1) = "АЗЩЯ"

OUT21 := frw_S_Caesar(IN1, K2) = "СЮАЖ"

IN2 := "ВЛОГ" OUT12 := frw_S_Caesar(IN2, K1) = "БЗЩЧ"

OUT22 := frw_S_Caesar(IN2, K2) = "ТЮА_"

INr11 := inv_S_Caesar(OUT11, K1) = "БЛОК" INe11 := inv_S_Caesar(OUT11, K2) = "ПХЗВ"

INr21 := inv_S_Caesar(OUT21, K2) = "БЛОК" INe21 := inv_S_Caesar(OUT21, K1) = "ТБХТ"

INr12 := inv_S_Caesar(OUT12, K1) = "ВЛОГ" INe12 := inv_S_Caesar(OUT12, K2) = "СХЗБ"

INr22 := inv_S_Caesar(OUT22, K2) = "ВЛОГ" INe22 := inv_S_Caesar(OUT22, K1) = "УБХЛ"

ПРИЛОЖЕНИЕ - ТРИТИМУС



ИНЖЕНЕРНАЯ ШКОЛА
ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ
И РОБОТОТЕХНИКИ

Построение алфавита для шифра Тритемуса

```
Thrithemus_table(KEY_IN) := || out ← ""
                             || A ← "АБВГДЕЖЗИЙКЛМНОПРСТУФХЦЧШЩЬЪЭЮЯ_"
                             || for i ∈ 0, 1..strlen(KEY_IN) - 1
                             ||   || tmp ← substr(KEY_IN, i, 1)
                             ||   || if strlen(out) < 32
                             ||   ||   || while search(out, tmp, 0) ≠ -1
                             ||   ||   ||   || tmp ← num2sym(mod(sym2num(tmp) + 1, 32))
                             ||   ||   || out ← concat(out, tmp)
                             || for i ∈ 0, 1..31
                             ||   || tmp ← substr(A, i, 1)
                             ||   || if search(out, tmp, 0) = -1
                             ||   ||   || out ← concat(out, tmp)
                             || out
```

K1 := "ДИНОЗАВР_ЗАУРОПОД"

Thrithemus_table(K1) = "ДИНОЗАВР_ЙБУСПТФЕГЖКЛМХЦЧШЩЬЪЭЮЯ"

K2 := "ГАМЕЛЬНСКИЙ_АНТИКВАР"

Thrithemus_table(K2) = "ГАМЕЛЬНСКИЙ_БОТПРВДУЖЗФХЦЧШЩЬЪЭЮЯ"

K3 := "ГАРРИ_ПОТТЕР_И_ФИЛОСОФСКИЙ_КАМЕНЬ"

Thrithemus_table(K3) = "ГАРСИ_ПОТУЕФБЙВХКЛЦЧШЩЬМНЬДЭЖЮЗЯ"

ПРИЛОЖЕНИЕ - ТРИТИМУС



ИНЖЕНЕРНАЯ ШКОЛА
ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ
И РОБОТОТЕХНИКИ

Простой шифр Тритимуса

```
frw_Trithemus(TXT_IN, KEY_TABLE) := || out ← ""
                                     || key_table ← KEY_TABLE
                                     || for i ∈ 0, 1..strlen(TXT_IN) - 1
                                     ||   || tmp ← substr(TXT_IN, i, 1)
                                     ||   || pos ← search(key_table, tmp, 0)
                                     ||   || csym ← substr(key_table, mod(pos + 8, 32), 1)
                                     ||   || out ← concat(out, csym)
                                     || out
```

IN := "ГОЛОВНОЙ_ОФИС" K1 := "ЧЕРНОСОТЕНЦЫ"
 K2 := "АБВГД" K3 := "А"

OUT1 := frw_Trithemus(IN, Thrithemus_table(K1)) = "ФАЮАМЫАБТА_ЩБ"

OUT2 := frw_Trithemus(IN, Thrithemus_table(K2)) = "ЛЦУЦКХЦСЗЦЭРЦ"

```
inv_Trithemus(TXT_IN, KEY_TABLE) := || out ← ""
                                     || key_table ← KEY_TABLE
                                     || for i ∈ 0, 1..strlen(TXT_IN) - 1
                                     ||   || tmp ← substr(TXT_IN, i, 1)
                                     ||   || pos ← search(key_table, tmp, 0)
                                     ||   || csym ← substr(key_table, mod(32 + pos - 8, 32), 1)
                                     ||   || out ← concat(out, csym)
                                     || out
```

IN1 := inv_Trithemus(OUT1, Thrithemus_table(K1)) = "ГОЛОВНОЙ_ОФИС"

IN2 := inv_Trithemus(OUT2, Thrithemus_table(K3)) = "ГОЛОВНОЙ_ОФИС"

ПРИЛОЖЕНИЕ - ТРИТИМУС



ИНЖЕНЕРНАЯ ШКОЛА
ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ
И РОБОТОТЕХНИКИ

Сдвиг ключевой таблицы для шифра
Тритимуса

```
shift_Trithemus(TABLE_IN, sym_in, bias_in) := || s ← sym_in
|| str ← substr(TABLE_IN, bias_in, 32 - bias_in)
|| rem ← substr(TABLE_IN, 0, bias_in)
|| while search(rem, s, 0) ≠ -1
|| || s ← num2sym(mod(sym2num(s) + 1, 32))
|| x ← search(str, s, 0)
|| str ← concat(substr(str, 0, x), substr(str, x + 1, 32 - x - 1))
|| TABLE_OUT ← concat(s, rem, str)
|| TABLE_OUT
```

$K := \text{"ХОРОШО_БЫТЬ_ВАМИ"}$

$KT := \text{Thrithemus_table}(K)$

$KT = \text{"ХОРПШС_БЫТЬАВГМИДЕЖЗЙКЛНУФЦЩЭЮЯ"}$

$K1 := \text{shift_Trithemus}(KT, \text{"Х"}, 16) = \text{"ЦХОРПШС_БЫТЬАВГМИДЕЖЗЙКЛНУФЦЩЭЮЯ"}$

$K2 := \text{shift_Trithemus}(KT, \text{"Х"}, 18) = \text{"ЦХОРПШС_БЫТЬАВГМИДЕЖЗЙКЛНУФЦЩЭЮЯ"}$

$K3 := \text{shift_Trithemus}(KT, \text{"Я"}, 16) = \text{"ЯХОРПШС_БЫТЬАВГМИДЕЖЗЙКЛНУФЦЩЭЮЯ"}$

$K4 := \text{shift_Trithemus}(KT, \text{"А"}, 8) = \text{"АХОРПШС_БЫТЬВГМИДЕЖЗЙКЛНУФЦЩЭЮЯ"}$

$K5 := \text{shift_Trithemus}(KT, \text{"А"}, 23) = \text{"НХОРПШС_БЫТЬАВГМИДЕЖЗЙКЛУФЦЩЭЮЯ"}$

$K6 := \text{shift_Trithemus}(KT, \text{"А"}, 30) = \text{"ЮХОРПШС_БЫТЬАВГМИДЕЖЗЙКЛНУФЦЩЭЯ"}$

$K7 := \text{shift_Trithemus}(KT, \text{"А"}, 1) = \text{"АХОРПШС_БЫТЬВГМИДЕЖЗЙКЛНУФЦЩЭЮЯ"}$

$\text{search}(\text{substr}(KT, 16, 32 - 16), \text{"Я"}, 0) = 15$

$\text{strlen}(\text{shift_Trithemus}(KT, \text{"А"}, 8)) = 32$

ПРИЛОЖЕНИЕ - ТРИТИМУС



ИНЖЕНЕРНАЯ ШКОЛА
ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ
И РОБОТОТЕХНИКИ

Полиалфавитный шифр Тритимуса

```
frw_poly_Trithemus(TXT_IN, KEY_IN) :=  
  out ← ""  
  key_table ← Thrithemus_table(KEY_IN)  
  key_array ← text2array(KEY_IN)  
  for i ∈ 0, 1..strlen(TXT_IN) - 1  
  |  
  | k ← mod(i, strlen(KEY_IN))  
  | b ← mod(strlen(KEY_IN) + i, 32)  
  | tmp ← substr(TXT_IN, i, 1)  
  | csym ← frw_Trithemus(tmp, key_table)  
  | key_table ← shift_Trithemus(key_table, num2sym(key_arrayk), b)  
  |  
  | out ← concat(out, csym)  
  out  
  
inv_poly_Trithemus(TXT_IN, KEY_IN) :=  
  out ← ""  
  key_table ← Thrithemus_table(KEY_IN)  
  key_array ← text2array(KEY_IN)  
  for i ∈ 0, 1..strlen(TXT_IN) - 1  
  |  
  | k ← mod(i, strlen(KEY_IN))  
  | b ← mod(strlen(KEY_IN) + i, 32)  
  | tmp ← substr(TXT_IN, i, 1)  
  | csym ← inv_Trithemus(tmp, key_table)  
  | key_table ← shift_Trithemus(key_table, num2sym(key_arrayk), b)  
  |  
  | out ← concat(out, csym)  
  out
```

K1 := "АББАТ_ТРИТИМУС" K2 := "БАБАТ_ТРИТИМУС" K3 := "ТРИТИМУС_АББАТ"

IN1 := "ОТКРЫТЫЙ_ТЕКСТ" IN2 := "ДЛИННЫЙ_ОТКРЫТЫЙ_ТЕКСТ"

OUT0 := frw_poly_Trithemus(IN1, K1) = "ЭХЩҚДХЖШСХУВНХ"
OUT1 := frw_poly_Trithemus(IN1, K2) = "ЭХЩҚДХЖШСХУВНХ"
OUT3 := frw_poly_Trithemus(IN2, K1) = "ПЫКБЬЕЧСШХВПЛХЦЬСХУВЫН"
OUT4 := frw_poly_Trithemus(IN2, K3) = "ПЬБЮЯЧГЖН_МАТ_ТГЮ_ЧФИВ"

INr0 := inv_poly_Trithemus(OUT0, K1) = "ОТКРЫТЫЙ_ТЕКСТ"
INr1 := inv_poly_Trithemus(OUT1, K2) = "ОТКРЫТЫЙ_ТЕКСТ"
INr2 := inv_poly_Trithemus(OUT1, K1) = "ОТКРЫТЫЙ_ТЕКСТ"
INr5 := inv_poly_Trithemus(OUT3, K3) = "ДКБЗЖСЬЦЬМУ_ЯМДБЦМЛЮЮЬ"
INr6 := inv_poly_Trithemus(OUT4, K3) = "ДЛИННЫЙ_ОТКРЫТЫЙ_ТЕКСТ"

ПРИЛОЖЕНИЕ - ТРИТИМУС



ИНЖЕНЕРНАЯ ШКОЛА
ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ
И РОБОТОТЕХНИКИ

S-блоки для шифра Тритимуса

```
frw_S_Trithemus(BLOCK_IN, KEY_IN) := || out ← "input_error"  
|| if (strlen(BLOCK_IN) = 4) ∧ (strlen(KEY_IN) = 16)  
|| || out ← frw_poly_Trithemus(BLOCK_IN, KEY_IN)  
|| out
```

```
inv_S_Trithemus(BLOCK_IN, KEY_IN) := || out ← "input_error"  
|| if (strlen(BLOCK_IN) = 4) ∧ (strlen(KEY_IN) = 16)  
|| || out ← inv_poly_Trithemus(BLOCK_IN, KEY_IN)  
|| out
```

K1 := "НЕТ_ЗВЕЗД_В_НОЧИ"

K2 := "ХОРОШО_БЫТЬ_ВАМИ"

IN1 := "БЛОК" OUT1 := frw_S_Trithemus(IN1, K1) = "РЩФЖ"

IN2 := "БРОК" OUT2 := frw_S_Trithemus(IN2, K1) = "РИФЖ"

IN1 := "БЛОК" OUT1 := frw_S_Trithemus(IN1, K2) = "ИЯТФ"

IN2 := "БРОК" OUT2 := frw_S_Trithemus(IN2, K2) = "ИЬТФ"

INr1 := inv_S_Trithemus(OUT1, K1) = "БЛОК"

INr2 := inv_S_Trithemus(OUT2, K1) = "БРОК"

INr1 := inv_S_Trithemus(OUT1, K1) = "ЯХЬБ"

INr2 := inv_S_Trithemus(OUT2, K1) = "ЯСЬБ"

INr11 := inv_S_Trithemus(OUT1, K2) = "ЦЗДЬ"

INr12 := inv_S_Trithemus(OUT2, K2) = "ЦБДЬ"

INr11 := inv_S_Trithemus(OUT1, K2) = "БЛОК"

INr12 := inv_S_Trithemus(OUT2, K2) = "БРОК"

ПРИЛОЖЕНИЕ - ОБЩЕЕ



Дополнительная операция для усиления S-блоков (прямая и обратная)

```

frw_merge_block(BLOCK_IN, KEY_IN) :=
    out ← "input_error"
    
$$M \leftarrow \begin{bmatrix} 0 \\ 1 \\ 2 \\ 3 \end{bmatrix}$$

    if (strlen(BLOCK_IN) = 4) ∧ (strlen(KEY_IN) = 16)
        array ← text2array(KEY_IN)
        for i ∈ 0, 1..15
            
$$sum \leftarrow \text{mod}(24 + sum + (-1)^i \cdot array_i, 24)$$

        for k ∈ 0, 1..2
            
$$t \leftarrow \text{mod}(sum, 4 - k)$$

            
$$sum \leftarrow \frac{sum - t}{4 - k}$$

            tmp ← Mk
            Mk ← Mk+t
            Mk+t ← tmp
        inp ← text2array(BLOCK_IN)
        for j ∈ 0, 1..3
            
$$b \leftarrow M_{\text{mod}(1+j, 4)}$$

            
$$a \leftarrow M_{\text{mod}(j, 4)}$$

            
$$inp_b \leftarrow \text{mod}(inp_b + inp_a, 32)$$

        out ← array2text(inp)
    out
    
```

```

inv_merge_block(BLOCK_IN, KEY_IN) :=
    out ← "input_error"
    
$$M \leftarrow \begin{bmatrix} 0 \\ 1 \\ 2 \\ 3 \end{bmatrix}$$

    if (strlen(BLOCK_IN) = 4) ∧ (strlen(KEY_IN) = 16)
        array ← text2array(KEY_IN)
        for i ∈ 0, 1..15
            
$$sum \leftarrow \text{mod}(24 + sum + (-1)^i \cdot array_i, 24)$$

        for k ∈ 0, 1..2
            
$$t \leftarrow \text{mod}(sum, 4 - k)$$

            
$$sum \leftarrow \frac{sum - t}{4 - k}$$

            tmp ← Mk
            Mk ← Mk+t
            Mk+t ← tmp
        inp ← text2array(BLOCK_IN)
        for j ∈ 3, 2..0
            
$$b \leftarrow M_{\text{mod}(1+j, 4)}$$

            
$$a \leftarrow M_{\text{mod}(j, 4)}$$

            
$$inp_b \leftarrow \text{mod}(32 + inp_b - inp_a, 32)$$

        out ← array2text(inp)
    out
    
```

ПРИЛОЖЕНИЕ - ОБЩЕЕ



ИНЖЕНЕРНАЯ ШКОЛА
ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ
И РОБОТОТЕХНИКИ

$K1 := \text{"ХОРОШО_ВЫТЬ_ВАМИ"}$ $IN1 := \text{"БЛОК"}$
 $K2 := \text{"ХОРОШО_ВЫТЬ_ВАМИ"}$ $IN2 := \text{"БРОК"}$
 $K3 := \text{"ХОРОШО_ВЫТЬ_БАМИ"}$

$OUT1 := \text{frw_merge_block}(IN1, K1) = \text{"БЗЦЩ"}$
 $OUT2 := \text{frw_merge_block}(IN2, K1) = \text{"БМЬЩ"}$
 $OUT3 := \text{frw_merge_block}(IN1, K2) = \text{"МЗБТ"}$
 $OUT4 := \text{frw_merge_block}(IN2, K2) = \text{"ММЬЧ"}$

$\text{inv_merge_block}(OUT1, K1) = \text{"БЛОК"}$
 $\text{inv_merge_block}(OUT2, K1) = \text{"БРОК"}$
 $\text{inv_merge_block}(OUT3, K1) = \text{"ЩЫУЯ"}$
 $\text{inv_merge_block}(OUT4, K1) = \text{"Ф_ОИ"}$

$\text{inv_merge_block}(OUT1, K2) = \text{"ЙРЫС"}$
 $\text{inv_merge_block}(OUT2, K2) = \text{"ОР_М"}$
 $\text{inv_merge_block}(OUT3, K2) = \text{"БЛОК"}$
 $\text{inv_merge_block}(OUT4, K2) = \text{"БРОК"}$

$\text{inv_merge_block}(OUT1, K3) = \text{"ДЛЭН"}$
 $\text{inv_merge_block}(OUT2, K3) = \text{"_РБИ"}$
 $\text{inv_merge_block}(OUT3, K3) = \text{"РЫИЧ"}$
 $\text{inv_merge_block}(OUT4, K3) = \text{"Р_ГЧ"}$

Модификация S-блоков для шифра Цезаря

$\text{frw_S_CaesarM}(BLOCK_IN, KEY_IN) := \left\{ \begin{array}{l} tmp \leftarrow \text{frw_merge_block}(BLOCK_IN, KEY_IN) \\ tmp \leftarrow \text{frw_S_Caesar}(tmp, KEY_IN) \\ out \leftarrow \text{frw_merge_block}(tmp, KEY_IN) \end{array} \right\}$

$\text{inv_S_CaesarM}(BLOCK_IN, KEY_IN) := \left\{ \begin{array}{l} tmp \leftarrow \text{inv_merge_block}(BLOCK_IN, KEY_IN) \\ tmp \leftarrow \text{inv_S_Caesar}(tmp, KEY_IN) \\ out \leftarrow \text{inv_merge_block}(tmp, KEY_IN) \end{array} \right\}$

$OUT1 := \text{frw_S_CaesarM}(IN1, K1) = \text{"УЫ_Ш"}$ $\text{inv_S_CaesarM}(OUT1, K1) = \text{"БЛОК"}$

Модификация S-блоков для шифра Тритимуса

$\text{frw_S_TrithemusM}(BLOCK_IN, KEY_IN) := \left\{ \begin{array}{l} tmp \leftarrow \text{inv_merge_block}(BLOCK_IN, KEY_IN) \\ tmp \leftarrow \text{frw_S_Trithemus}(tmp, KEY_IN) \\ out \leftarrow \text{frw_merge_block}(tmp, KEY_IN) \end{array} \right\}$

$\text{inv_S_TrithemusM}(BLOCK_IN, KEY_IN) := \left\{ \begin{array}{l} tmp \leftarrow \text{inv_merge_block}(BLOCK_IN, KEY_IN) \\ tmp \leftarrow \text{inv_S_Trithemus}(tmp, KEY_IN) \\ out \leftarrow \text{frw_merge_block}(tmp, KEY_IN) \end{array} \right\}$

$OUT1 := \text{frw_S_TrithemusM}(IN1, K1) = \text{"ЧФЮЖ"}$ $\text{inv_S_TrithemusM}(OUT1, K1) = \text{"БЛОК"}$

ПРИЛОЖЕНИЕ - ТЕСТЫ



Тест на изменение входа (близкие входы)

$IN1 := \text{"ОРЕХ"}$ $IN2 := \text{"ОПЕХ"}$ $IN3 := \text{"ОПЕФ"}$ $KEY1 := \text{"ХОРОШО_БЫТЬ_ВАМИ"}$ $KEY2 := \text{"МОЛЧАНИЕ_ЗОЛОТО_"}$

$frw_S_Caesar(IN1, KEY1) = \text{"НМРЙ"}$	$frw_S_Caesar(IN1, KEY2) = \text{"ОГИЧ"}$
$frw_S_Caesar(IN2, KEY1) = \text{"НЛРЙ"}$	$frw_S_Caesar(IN2, KEY2) = \text{"ОВИЧ"}$
$frw_S_Caesar(IN3, KEY1) = \text{"НЛРИ"}$	$frw_S_Caesar(IN3, KEY2) = \text{"ОВИЦ"}$

Изменение 1 символа на входе, меняет 1 соответствующий символ на выходе (плохо)

$frw_S_Trithemus(IN1, KEY1) = \text{"ТЬЧЫ"}$	$frw_S_Trithemus(IN1, KEY2) = \text{"ЗЖБИ"}$
$frw_S_Trithemus(IN2, KEY1) = \text{"ТАЧЫ"}$	$frw_S_Trithemus(IN2, KEY2) = \text{"ЗДБИ"}$
$frw_S_Trithemus(IN3, KEY1) = \text{"ТАЧС"}$	$frw_S_Trithemus(IN3, KEY2) = \text{"ЗДБЕ"}$

Изменение 1 символа на входе, меняет 1 соответствующий символ на выходе (плохо)

$frw_merge_block(IN1, KEY1) = \text{"ДЬКС"}$	$frw_merge_block(IN1, KEY2) = \text{"ФЕБЬ"}$
$frw_merge_block(IN2, KEY1) = \text{"ДЫКР"}$	$frw_merge_block(IN2, KEY2) = \text{"ФДАЫ"}$
$frw_merge_block(IN3, KEY1) = \text{"ГЩЙО"}$	$frw_merge_block(IN3, KEY2) = \text{"ФД_Щ"}$

Изменение 1 символа на входе, иногда меняет несколько символов на выходе (не хорошо)

ПРИЛОЖЕНИЕ - ТЕСТЫ



ИНЖЕНЕРНАЯ ШКОЛА
ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ
И РОБОТОТЕХНИКИ

Тест на изменение входа (ротация входа)

$IN1 := \text{"ОРЕХ"}$ $IN2 := \text{"РОЕХ"}$ $IN3 := \text{"РЕХО"}$ $KEY1 := \text{"ХОРОШО_БЫТЬ_ВАМИ"}$ $KEY2 := \text{"МОЛЧАНИЕ_ЗОЛОТО_"}$

$frw_S_Caesar(IN1, KEY1) = \text{"НМРЙ"}$	$frw_S_Caesar(IN1, KEY2) = \text{"ОГИЧ"}$
$frw_S_Caesar(IN2, KEY1) = \text{"ПКРЙ"}$	$frw_S_Caesar(IN2, KEY2) = \text{"РБИЧ"}$
$frw_S_Caesar(IN3, KEY1) = \text{"ПБАВ"}$	$frw_S_Caesar(IN3, KEY2) = \text{"РШШР"}$

Перестановка (сдвиг) входа не приводит к перестановке (сдвигу) выхода (нормально)

$frw_S_Trithemus(IN1, KEY1) = \text{"ТЬЧЫ"}$	$frw_S_Trithemus(IN1, KEY2) = \text{"ЗЖБИ"}$
$frw_S_Trithemus(IN2, KEY1) = \text{"ЬТЧЫ"}$	$frw_S_Trithemus(IN2, KEY2) = \text{"ЖЗБИ"}$
$frw_S_Trithemus(IN3, KEY1) = \text{"ЬФЫТ"}$	$frw_S_Trithemus(IN3, KEY2) = \text{"ЖБИЗ"}$

Перестановка (сдвиг) входа приводит к перестановке (сдвигу) выхода (плохо)

$frw_merge_block(IN1, KEY1) = \text{"ДЬКС"}$	$frw_merge_block(IN1, KEY2) = \text{"ФЕБЬ"}$
$frw_merge_block(IN2, KEY1) = \text{"ЖЬМС"}$	$frw_merge_block(IN2, KEY2) = \text{"ЦЕБЬ"}$
$frw_merge_block(IN3, KEY1) = \text{"_БХК"}$	$frw_merge_block(IN3, KEY2) = \text{"ЖМСЬ"}$

Перестановка (сдвиг) входа не приводит к перестановке (сдвигу) выхода (нормально)

ПРИЛОЖЕНИЕ - ТЕСТЫ



Тест на изменение входа (линейность операции)

$IN4 := add_txt(IN1, IN3) = \text{"_цбд"}$

$add_txt(fr_S_Caesar(IN1, KEY1), fr_S_Caesar(IN3, KEY1)) = \text{"ЮОСМ"}$ $fr_S_Caesar(IN4, KEY1) = \text{"ЯТЖШ"}$

$add_txt(fr_S_Caesar(IN1, KEY2), fr_S_Caesar(IN3, KEY2)) = \text{"_ЭБИ"}$ $fr_S_Caesar(IN4, KEY2) = \text{"_ЙЯЖ"}$

Сумма выходов не похожа на выход суммы (нормально)

$add_txt(fr_S_Trithemus(IN1, KEY1), fr_S_Trithemus(IN3, KEY1)) = \text{"ОПТН"}$ $fr_S_Trithemus(IN4, KEY1) = \text{"МБЖЧ"}$

$add_txt(fr_S_Trithemus(IN1, KEY2), fr_S_Trithemus(IN3, KEY2)) = \text{"ОИКР"}$ $fr_S_Trithemus(IN4, KEY2) = \text{"ВФЛЭ"}$

Сумма выходов не похожа на выход суммы (нормально)

$add_txt(fr_merge_block(IN1, KEY1), fr_merge_block(IN3, KEY1)) = \text{"ДЧАЭ"}$ $fr_merge_block(IN4, KEY1) = \text{"ДЧАЭ"}$

$add_txt(fr_merge_block(IN1, KEY2), fr_merge_block(IN3, KEY2)) = \text{"бТУЧ"}$ $fr_merge_block(IN4, KEY2) = \text{"бТУЧ"}$

Сумма выходов тождественны выходу суммы (очень плохо)

ПРИЛОЖЕНИЕ - ТЕСТЫ



Тест на изменение ключа (близкие ключи)

$IN1 := \text{"ОРЕХ"}$ $IN2 := \text{"УЛАН"}$ $KEY1 := \text{"ЖАРОВЫНОСЛИВОСТЬ"}$ $KEY2 := \text{"ЖБРОВЫНОСЛИВОСТЬ"}$ $KEY3 := \text{"ЖБРОВЫНОСКИВОСТЬ"}$

$frw_S_Caesar(IN1, KEY1) = \text{"ОТЧЮ"}$	$frw_S_Caesar(IN2, KEY1) = \text{"УНТХ"}$
$frw_S_Caesar(IN1, KEY2) = \text{"ОСЦ_"} $	$frw_S_Caesar(IN2, KEY2) = \text{"УМСЧ"}$
$frw_S_Caesar(IN1, KEY3) = \text{"ОПФЬ"}$	$frw_S_Caesar(IN2, KEY3) = \text{"УКПУ"}$

Изменение 1 символа в ключе, иногда меняет несколько символов в выходе (нехорошо)

$frw_S_Trithemus(IN1, KEY1) = \text{"ГИЩЫ"}$	$frw_S_Trithemus(IN2, KEY1) = \text{"КДЛФ"}$
$frw_S_Trithemus(IN1, KEY2) = \text{"ГИЩЫ"}$	$frw_S_Trithemus(IN2, KEY2) = \text{"КДШФ"}$
$frw_S_Trithemus(IN1, KEY3) = \text{"ГИЩЫ"}$	$frw_S_Trithemus(IN2, KEY3) = \text{"ЛЮШФ"}$

Изменение 1 символа в ключе, иногда меняет несколько символов в выходе,
но есть есть уязвимые входы и слабые ключи (очень плохо)

$frw_merge_block(IN1, KEY1) = \text{"ЬМЬС"}$	$frw_merge_block(IN2, KEY1) = \text{"ОЫОЭ"}$
$frw_merge_block(IN1, KEY2) = \text{"ФЕБЬ"}$	$frw_merge_block(IN2, KEY2) = \text{"ФАПО"}$
$frw_merge_block(IN1, KEY3) = \text{"ЬМЬС"}$	$frw_merge_block(IN2, KEY3) = \text{"ОЫОЭ"}$

Влияет только сумма символов в ключе (никуда не годится)

ПРИЛОЖЕНИЕ - ТЕСТЫ



Тест на изменение ключа (ротация ключа)

$IN1 := \text{“ОРЕХ”}$ $IN2 := \text{“УЛАН”}$ $KEY1 := \text{“ЖАРОВЫНОСЛИВОСТЬ”}$ $KEY2 := \text{“АЖРОВЫНОСЛИВОСТЬ”}$ $KEY3 := \text{“ТЬЖАРОВЫНОСЛИВОС”}$

$frw_S_Caesar(IN1, KEY1) = \text{“ОТЧЮ”}$	$frw_S_Caesar(IN2, KEY1) = \text{“УНТХ”}$
$frw_S_Caesar(IN1, KEY2) = \text{“ИЖЕЮ”}$	$frw_S_Caesar(IN2, KEY2) = \text{“НБАХ”}$
$frw_S_Caesar(IN1, KEY3) = \text{“ШТБЙ”}$	$frw_S_Caesar(IN2, KEY3) = \text{“ЮНЭБ”}$

Перестановка символов в ключе не приводит к перестановке символов в выходе (нормально)

$frw_S_Trithemus(IN1, KEY1) = \text{“ГИЩЫ”}$	$frw_S_Trithemus(IN2, KEY1) = \text{“КДЛФ”}$
$frw_S_Trithemus(IN1, KEY2) = \text{“ГИЩЫ”}$	$frw_S_Trithemus(IN2, KEY2) = \text{“КЕСФ”}$
$frw_S_Trithemus(IN1, KEY3) = \text{“ГИЩЫ”}$	$frw_S_Trithemus(IN2, KEY3) = \text{“МЗЛБ”}$

Перестановка символов в ключе не приводит к перестановке символов в выходе,
но есть неудачные (очень плохо)

$frw_merge_block(IN1, KEY1) = \text{“ЬМЬС”}$	$frw_merge_block(IN2, KEY1) = \text{“ОЫОЭ”}$
$frw_merge_block(IN1, KEY2) = \text{“КЬЬС”}$	$frw_merge_block(IN2, KEY2) = \text{“ВООЭ”}$
$frw_merge_block(IN1, KEY3) = \text{“ЬМЬС”}$	$frw_merge_block(IN2, KEY3) = \text{“ОЫОЭ”}$

Влияет только сумма символов в ключе (никуда не годится)

ПРИЛОЖЕНИЕ - ТЕСТЫ



Тест на изменение ключа (линейность операции)

$KEY3 := add_txt(KEY1, KEY2) = \text{“3ЗБЮЕХЬЮГЧСЕЮГЕЧ”}$

$add_txt(fr_S_Caesar(IN1, KEY1), fr_S_Caesar(IN1, KEY2)) = \text{“ЧЩЮЬ”}$ $fr_S_Caesar(IN1, KEY3) = \text{“ИА_Х”}$

$add_txt(fr_S_Caesar(IN2, KEY1), fr_S_Caesar(IN2, KEY2)) = \text{“БПУЛ”}$ $fr_S_Caesar(IN2, KEY3) = \text{“НЬЫН”}$

Сумма выходов не похожа на выход суммы (нормально)

$add_txt(fr_S_Trithemus(IN1, KEY1), fr_S_Trithemus(IN1, KEY2)) = \text{“ЗСУХ”}$ $fr_S_Trithemus(IN1, KEY3) = \text{“ЫК_Д”}$

$add_txt(fr_S_Trithemus(IN2, KEY1), fr_S_Trithemus(IN2, KEY1)) = \text{“ХЙЧЙ”}$ $fr_S_Trithemus(IN2, KEY3) = \text{“БФУЩ”}$

Сумма выходов не похожа на выход суммы (нормально)

$add_txt(fr_merge_block(IN1, KEY1), fr_merge_block(IN1, KEY2)) = \text{“ЖИЧГ”}$ $fr_merge_block(IN1, KEY3) = \text{“ЕЦБЬ”}$

$add_txt(fr_merge_block(IN2, KEY1), fr_merge_block(IN2, KEY2)) = \text{“СЙЮЩ”}$ $fr_merge_block(IN2, KEY3) = \text{“АМПО”}$

Сумма выходов не похожа на выход суммы (нормально)

ПРИЛОЖЕНИЕ - ТЕСТЫ



ИНЖЕНЕРНАЯ ШКОЛА
ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ
И РОБОТОТЕХНИКИ

Тест на изменение входа (близкие входы)

$IN1 := \text{"ОРЕХ"}$ $IN2 := \text{"ОПЕХ"}$ $IN3 := \text{"ОПЕФ"}$ $KEY1 := \text{"ХОРОШО_БЫТЬ_ВАМИ"}$ $KEY2 := \text{"МОЛЧАНИЕ_ЗОЛОТО_"}$

$frw_S_CaesarM(IN1, KEY1) = \text{"ЙЧ_Ю"}$ $frw_S_CaesarM(IN1, KEY2) = \text{"ЩТХР"}$
 $frw_S_CaesarM(IN2, KEY1) = \text{"ИХЯЫ"}$ $frw_S_CaesarM(IN2, KEY2) = \text{"ШРСН"}$
 $frw_S_CaesarM(IN3, KEY1) = \text{"ЕРЫУ"}$ $frw_S_CaesarM(IN3, KEY2) = \text{"ЧПОЛ"}$

Изменение 1 символа на входе, меняет все символы на выходе (нормально)

$frw_S_TrithemusM(IN1, KEY1) = \text{"ССУЙ"}$ $frw_S_TrithemusM(IN1, KEY2) = \text{"КБГЯ"}$
 $frw_S_TrithemusM(IN2, KEY1) = \text{"ЗЖЙА"}$ $frw_S_TrithemusM(IN2, KEY2) = \text{"КЭГЯ"}$
 $frw_S_TrithemusM(IN3, KEY1) = \text{"СРУИ"}$ $frw_S_TrithemusM(IN3, KEY2) = \text{"ПБЕЯ"}$

Изменение 1 символа на входе, меняет многие символы на выходе (почти нормально)

ПРИЛОЖЕНИЕ - ТЕСТЫ



ИНЖЕНЕРНАЯ ШКОЛА
ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ
И РОБОТОТЕХНИКИ

Тест на изменение входа (ротация входа)

$IN1 := \text{"OPEX"}$ $IN2 := \text{"POEX"}$ $IN3 := \text{"PEXO"}$ $KEY1 := \text{"ХОРОШО_БЫТЬ_ВАМИ"}$ $KEY2 := \text{"МОЛЧАНИЕ_ЗОЛОТО_}"$

$frw_S_CaesarM(IN1, KEY1) = \text{"ЙЧ_Ю"}$ $frw_S_CaesarM(IN1, KEY2) = \text{"ЩТХР"}$
 $frw_S_CaesarM(IN2, KEY1) = \text{"ЛЬГБ"}$ $frw_S_CaesarM(IN2, KEY2) = \text{"ЬФЧТ"}$
 $frw_S_CaesarM(IN3, KEY1) = \text{"ЮЦЯХ"}$ $frw_S_CaesarM(IN3, KEY2) = \text{"ЬЬОЩ"}$

Перестановка (сдвиг) входа не приводит к перестановке (сдвигу) выхода (нормально)

$frw_S_TrithemusM(IN1, KEY1) = \text{"ССУЙ"}$ $frw_S_TrithemusM(IN1, KEY2) = \text{"КБГЯ"}$
 $frw_S_TrithemusM(IN2, KEY1) = \text{"ЦЫМЧ"}$ $frw_S_TrithemusM(IN2, KEY2) = \text{"ЦДЗВ"}$
 $frw_S_TrithemusM(IN3, KEY1) = \text{"ЕЬЫЙ"}$ $frw_S_TrithemusM(IN3, KEY2) = \text{"ШЯПТ"}$

Перестановка (сдвиг) входа не приводит к перестановке (сдвигу) выхода (нормально)

ПРИЛОЖЕНИЕ - ТЕСТЫ



ИНЖЕНЕРНАЯ ШКОЛА
ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ
И РОБОТОТЕХНИКИ

Тест на изменение входа (линейность операции)

$IN4 := add_txt(IN1, IN3) = \text{"_ЦБД"}$

$add_txt(fr_w_S_CaesarM(IN1, KEY1), fr_w_S_CaesarM(IN3, KEY1)) = \text{"ЗОЯУ"}$ $fr_w_S_CaesarM(IN4, KEY1) = \text{"ФФАЕ"}$

$add_txt(fr_w_S_Caesar(IN1, KEY2), fr_w_S_Caesar(IN3, KEY2)) = \text{"_ЭБИ"}$ $fr_w_S_CaesarM(IN4, KEY2) = \text{"ТШЙТ"}$

Сумма выходов не похоже на выход суммы (нормально)

$add_txt(fr_w_S_TrithemusM(IN1, KEY1), fr_w_S_TrithemusM(IN3, KEY1)) = \text{"ЧНОУ"}$ $fr_w_S_TrithemusM(IN4, KEY1) = \text{"ЫЖБЮ"}$

$add_txt(fr_w_S_TrithemusM(IN1, KEY2), fr_w_S_TrithemusM(IN3, KEY2)) = \text{"ГАУС"}$ $fr_w_S_TrithemusM(IN4, KEY2) = \text{"ЙЯЗС"}$

Сумма выходов не похоже на выход суммы (нормально)

ПРИЛОЖЕНИЕ - ТЕСТЫ



Тест на изменение ключа (близкие ключи)

$IN1 := \text{"ОРЕХ"}$ $IN2 := \text{"УЛАН"}$ $KEY1 := \text{"ЖАРОВЫНОСЛИВОСТЬ"}$ $KEY2 := \text{"ЖБРОВЫНОСЛИВОСТЬ"}$ $KEY3 := \text{"ЖБРОВЫНОСКИВОСТЬ"}$

$frw_S_CaesarM(IN1, KEY1) = \text{"ТЦЗМ"}$ $frw_S_CaesarM(IN2, KEY1) = \text{"СВЕЦ"}$
 $frw_S_CaesarM(IN1, KEY2) = \text{"ЗОЗФ"}$ $frw_S_CaesarM(IN2, KEY2) = \text{"ХЧСР"}$
 $frw_S_CaesarM(IN1, KEY3) = \text{"КОВВ"}$ $frw_S_CaesarM(IN2, KEY3) = \text{"ЙЫАМ"}$

Изменение 1 символа в ключе меняет несколько символов в выходе (нормально)

$frw_S_TrithemusM(IN1, KEY1) = \text{"НТФ_"}$ $frw_S_TrithemusM(IN2, KEY1) = \text{"РЦШЧ"}$
 $frw_S_TrithemusM(IN1, KEY2) = \text{"ЙХ_Г"}$ $frw_S_TrithemusM(IN2, KEY2) = \text{"ГКГЦ"}$
 $frw_S_TrithemusM(IN1, KEY3) = \text{"ФЩФЖ"}$ $frw_S_TrithemusM(IN2, KEY3) = \text{"ЧЮШЯ"}$

Изменение 1 символа в ключе меняет несколько символов в выходе, проблема с уязвимыми ключами частично решена (не так плохо)

ПРИЛОЖЕНИЕ - ТЕСТЫ



Тест на изменение ключа (ротация ключа)

$IN1 := \text{"ОРЕХ"}$ $IN2 := \text{"УЛАН"}$ $KEY1 := \text{"ЖАРОВЫНОСЛИВОСТЬ"}$ $KEY2 := \text{"АЖРОВЫНОСЛИВОСТЬ"}$ $KEY3 := \text{"ТЬЖАРОВЫНОСЛИВОС"}$

$frw_S_CaesarM(IN1, KEY1) = \text{"ТЦЗМ"}$ $frw_S_CaesarM(IN2, KEY1) = \text{"СВЕЦ"}$
 $frw_S_CaesarM(IN1, KEY2) = \text{"ЫМХЖ"}$ $frw_S_CaesarM(IN2, KEY2) = \text{"РХУЫ"}$
 $frw_S_CaesarM(IN1, KEY3) = \text{"ТМЮШ"}$ $frw_S_CaesarM(IN2, KEY3) = \text{"СШЬВ"}$

Перестановка символов в ключе не приводит к перестановке символов в выходе (нормально)

$frw_S_TrithemusM(IN1, KEY1) = \text{"НТФ_"} \quad frw_S_TrithemusM(IN2, KEY1) = \text{"РЦШЧ"}$
 $frw_S_TrithemusM(IN1, KEY2) = \text{"СБЛЬ"} \quad frw_S_TrithemusM(IN2, KEY2) = \text{"ЭЮТН"}$
 $frw_S_TrithemusM(IN1, KEY3) = \text{"РБВВ"} \quad frw_S_TrithemusM(IN2, KEY3) = \text{"ЖМНВ"}$

Перестановка символов в ключе не приводит к перестановке символов в выходе,
проблема с уязвимыми ключами частично решена (не так плохо)